Analysis of Heatpumps

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Leander Marius Bürkin

universität freiburg

Bachelor's Thesis

Submitted on **April 23, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Paulo Rosas** (4141122)
Examined by **Prof. Dr. Mathias Fleury**

Systems Control and Optimization Laboratory
Department of Microsystems Engineering (IMTEK)
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

1. Enter your data.

2. Add as many bigskips as needed.

Enter your data int the Declaration.

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, April 23, 2026

Place, date	Leander Marius Bürkin

Acknowledgements

Thanks to my supervisor Max Mustermensch.

Further thanks go to the corporate design team at the Albert-Ludwigs-Universität Freiburg and Leander Marius Bürkin for this LaTeX-template.

Abstract

Zusammenfassung

Contents

1 Einleitung **1**
1.1 Conjunctive Normal Form (CNF) 1
1.2 Warum SAT wichtig ist 2

2 Preliminaries **4**
2.1 Kissat 4
2.1.1 Kissat Architektur 4
2.1.2 Kissat-Konfigurationsvarianten 5
2.1.3 Messung der Solver-Leistung 5
2.2 Satzilla 5
2.2.1 Feature Extraction 6
2.2.2 Machine Learning Model 6
2.2.3 Training und Prediction Pipeline 7
2.2.4 Warum Satzilla nach wie vor von Bedeutung ist 8

3 Satzilla Outcome **9**
3.1 Validierung mit Fake Solvern 9
3.1.1 Design der fake Solver 9
3.1.2 Resultate 9
3.1.3 Interpretation 10
3.2 Trainieren auf echten Kissat Solvern 11
3.2.1 Dataset 11
3.2.2 Training Konfiguration 11
3.2.3 Testgenauigkeit 11
3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“ 11
3.3.1 Resultate 11
3.3.2 Interpretation 11
3.3.3 Kumulative Genauigkeitskurve 11
3.4 Feature Importance Analysis 11
3.4.1 Wichtigste Features 11
3.4.2 Konklusion 11
3.5 Per-Solver-Leistung 11
3.5.1 Interpretation 11
3.6 Vergleich mit einer zufälligen Referenzlinie 11
3.7 Praktischer Nutzen für die SAT-Lösung 11
3.8 Auswirkungen der Hinzufügung von fehenden LP-Features 11
3.8.1 Interpretation 11
3.9 Zusammenfassung der wichtigsten Ergebnisse 11
3.10 Konklusion 11

4 Results **12**

5 Discussion **13**

6 Conclusion **14**

6.1 Tables and References 15
6.2 Auto-complete in VSCode 15
6.3 ”, Quotations and Code 16
6.4 Clipboard Manager, Git and more 17
6.5 Glossary 17

Symbols **I**

List of Figures **II**

List of Tables **III**

List of Algorithms **IV**

Glossary **V**

References **VI**

Appendix **VII**

1 Einleitung

Das Erfüllbarkeitsproblem der Aussagenlogik (SAT, von englisch satisfabiality "Erfüllbarkeit") ist ein fundamentales Problem der Informatik mit kritischen Anwendungen in Software und Hardware Überprüfung.

1.1 Conjunctive Normal Form (CNF)

Bei einer Boolschen Formel, die aus durch UND- und ODER-Operatoren verbunden Variablen besteht, (AND $\wedge$, OR $\vee$, NOT $\neg$) geht es beim SAT-Problem um die Frage, ob es eine Zuorndung von Warheitswerten zu den Variablen gibt, bei der die gesamte Formel als Wahr ausgewertet wird.

Moderne SAT-Löser arbeiten in der Regel mit Formeln in konjunktiver Normalform (CNF). Eine CNF-Formel besteht aus:

- **Variablen**: Boolesche Variablen (z. B. x_1, x_2, ..., x_n)
- **Literale**: Eine Variable oder deren Negation (z. B. x_1 oder $\neg x_1$)
- **Klauseln**: Eine Disjunktion (ODER) von Literalen (z. B. $(x_1 \vee \neg x_2 \vee x_3)$)
- **Formel**: Eine Konjunktion (UND) von Satzteilen (z. B. $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$

Das CNF-Format ist sowohl ausdrucksstark (jede Boolesche Formel lässt sich in CNF umwandeln) als auch für algorithmisches Denken gut geeignet. Das folgende Beispiel zeigt eine einfache CNF-Datei:

```
p cnf 3 4
1 -2 0
-1 3 0
2 -3 0
-1 -2 -3 0
```

Dies stellt eine Formel mit 3 Variablen und 4 Klauseln dar. Die Zeile „p cnf“ gibt den Problemtyp und die Dimensionen an (3 Variablen, 4 Klauseln), und jede nachfolgende Zeile stellt eine Klausel dar, die mit 0 endet.

1.2 Warum SAT wichtig ist

Trotz seiner theoretischen Bedeutung als erstes NP-vollständiges Problem (Cook, 1971) hat SAT breite praktische Anwendung in den Bereichen Hardware-Verifikation, Software-Testing, Planung und künstliche Intelligenz gefunden.[3]

Moderne SAT-solver haben in den letzten zwei Jahrzenten bemerkenswerte fortschritte gemacht; Löser wie Kissat (Biere et al., 2020) sind in der Lage, industrielle Instanzen mit Millionen von Variablen zu lösen.[1]

SAT hat breite praktische Anwendung gefunden, da sich viele Probleme aus der realen Welt als SAT-Instanzen formulieren lassen:

- **Hardware-Verifikation**: Nachweis, dass sich digitale Schaltungen korrekt verhalten
- **Softwaretests**: Erstellung von Testfällen, die bestimmte Pfade abdecken
- **Planung und Terminierung**: Ermittlung von Handlungsabläufen, mit denen Ziele erreicht werden
- **Kryptoanalyse**: Knacken kryptografischer Systeme durch das Lösen logischer Beschränkungen
- **Bioinformatik**: Analyse genetischer Sequenzen und Proteinfaltung

Die praktische Bedeutung von SAT hat jahrzehntelange Forschungen zur Entwicklung effizienter SAT-Löser vorangetrieben.

Der praktische Erfolg der SAT-Lösung ist weitgehend auf die bemerkenswerte Leistungsfähigkeit von CDCL-Lösern (Conflict-Driven Clause Learning) zurückzuführen. In der SAT-Forschung ist jedoch bekannt, dass kein einzelner Löser bei allen Instanzen allen anderen überlegen ist. Verschiedene Solver weisen komplementäre Leistungsmerkmale auf – ein Solver, der bei einer Instanzfamilie extrem schnell ist, kann bei einer anderen schlecht abschneiden. Diese Beobachtung hat zur Entwicklung von Algorithmusauswahlsystemen geführt, die automatisch den für eine bestimmte Instanz am besten geeigneten Solver auswählen.

Diese Erkenntnis hat zur Entwicklung des portfoliabasierten SAT-Lösens geführt, bei dem eine Sammlung von Solvern gepflegt wird und ein Auswahlmechanismus entscheidet, welcher Solver für eine bestimmte Instanz eingesetzt wird. Die zugrunde liegende Idee ist einfach, aber wirkungsvoll: Wenn wir vorhersagen können, welcher Solver bei einer Instanz die beste Leistung erbringt, können wir erhebliche Rechenzeit einsparen,

indem wir Solver mit schlechter Leistung vermeiden.

2 Preliminaries

2.1 Kissat

Kissat ist ein hochmoderner SAT-Löser nach dem Prinzip des Conflict-Driven Clause Learning (CDCL), der von Armin Biere und seiner Forschungsgruppe an der Johannes Kepler Universität Linz entwickelt wurde. Seit seiner ersten Veröffentlichung im Jahr 2020 hat sich Kissat zu einem der erfolgreichsten Löser in den jüngsten SAT-Wettbewerben entwickelt, zahlreiche Medaillen gewonnen und bei einer Vielzahl von Benchmark-Familien eine außergewöhnliche Leistung gezeigt.

Der Name „Kissat“ leitet sich vom deutschen Wort „Kissat“ ab (eine spielerische Abwandlung von „kein SAT“) und spiegelt die Philosophie des Solvers wider, auf eine saubere und effiziente Implementierung zu setzen. Kissat ist in der Programmiersprache C geschrieben und legt Wert auf Einfachheit, Leistung und Reproduzierbarkeit.

2.1.1 Kissat Architektur

Kissat implementiert den CDCL-Algorithmus mit mehreren ausgefeilten Optimierungen:

Komponente	**Beschreibung**
Variablen Elimination	Entfernt Variablen durch auflösen der Klasuen, vereinfacht dadurch die Formel
Ausschluss durch Subsumption	Entfernt Klauseln, die logisch aus anderen abgeleitet werden können
Untersuchen	Durchsucht den Suchraum, um Ausbreitungswege zu finden
Zwischenspeicherung	Speichert erlernte Informationen und nutzt sie nach einem Neustart wieder
Mehrstufige Entscheidungsheuristik	Stellt zuletzt verwendete Variablen in den Vordergrund

Table 1: Kissat Optimierungen

2.1.2 Kissat-Konfigurationsvarianten

Kissat bietet zahlreiche Konfigurationsoptionen, mit denen sich sein Verhalten anpassen lässt. Die in dieser Arbeit verwendeten Varianten stellen unterschiedliche Kompromisse zwischen dem Aufwand für die Vorverarbeitung und der Rechengeschwindigkeit dar:

Variante	Haupteigenschaften
kissat_default	Standardkonfiguration mit ausgewogenen Einstellungen
kissat_noeliminate	Deaktiviert die Variableneliminierung (schnellere Vorverarbeitung, möglicherweise langsamere Lösung)
kissat_nostable	Deaktiviert den stabilen Modus (abweichende Strategie zur Variablenauswahl)
kissat_stable	Aktiviert den stabilen Modus für eine systematischere Suche
kissat_stable2	Erweiterter Stabilitätsmodus mit zusätzlichen Heuristiken
kissat_target2	Optimiert für eine bestimmte Problemklasse

Table 2: Kissat-Konfigurationsvarianten

2.1.3 Messung der Solver-Leistung

Die Leistung eines Solvers wird in der Regel anhand der Gesamtlaufzeit oder der CPU-Zeit gemessen. In dieser Arbeit verwende ich die vom Feld „process-time“ von Kissat gemeldete Laufzeit, die die zur Lösung der Instanz aufgewendeten CPU-Sekunden angibt. Zeitüberschreitungen (z. B. 3600 Sekunden, 7200 Sekunden) deuten darauf hin, dass der Solver die Berechnung nicht innerhalb der zugewiesenen Zeit abgeschlossen hat.

2.2 Satzilla

Satzilla (Xu et al., 2008) ist ein wegweisendes System, das maschinelles Lernen auf das Problem der Solverauswahl anwendet. Die zentrale Erkenntnis besteht darin, dass SAT-Instanzen durch Merkmale charakterisiert werden können, die die Leistung des Solvers vorhersagen. Durch das Lernen aus historischen Laufzeitdaten kann Satzilla den besten Solver für bisher unbekannte Instanzen auswählen.[4]

2.2.1 Feature Extraction

Satzilla extrahiert aus jeder CNF-Formel über 100 Merkmale, die in mehrere Kategorien unterteilt sind:

Kategorie	Features	Was sie messen
Basic	Anzahl der Variablen, Anzahl der Klauseln, Verhältnis	Größe und Dichte
Variable-Clause Graph (VCG)	Mittelwert, Entropie, Min/-Max	Verbindungsstruktur eines Graphen
Clause Graph (CG)	Klauselverbindungen, Cluster	Wechselwirkungen zwischen Klauseln
Variable Graph (VG)	Variable co-occurrence	Variable Beziehungen
Horn Features	Horn-Klausel-Bruch, Eigenschaften der Horn-Variablen	Besondere Struktur
Probing Features	Ausbreitungstiefe der Einheit, reduzierte Variablen	Eigenschaften des Suchraums
Local Search	Leistungskennzahlen von GSAT/SAPS	Die Landschaft der lokalen Suche
LP Features	Relaxationswerte der linearen Programmierung	Kontinuierliche Entspannung

Table 3: SAT Instance Features Categories

Diese Merkmale werden von einem speziellen Merkmalsextraktor (Merkmalsexekutivdatei) berechnet, der die CNF-Datei analysiert und eine CSV-Datei ausgibt, die eine Zeile pro Instanz enthält.

2.2.2 Machine Learning Model

Satzilla nutzt „Random Forest“ (Breiman, 2001) als Lernalgorithmus. „Random Forest“ ist ein Ensemble-Verfahren, bei dem mehrere Entscheidungsbäume erstellt und deren Vorhersagen zusammengefasst werden. Wesentliche Vorteile für diese Anwendung:

- Unterstützt gemischte Merkmalstypen (kontinuierlich, kategorial)
- Unempfindlich gegenüber irrelevanten Merkmalen (der Algorithmus ignoriert verrauschte Eingaben)
- Zeigt die Merkmalsbedeutung an (welche Merkmale am wichtigsten sind)
- Verhindert Überanpassung durch geeignete Hyperparameter

Im ursprünglichen Satzilla sagt das Modell die Laufzeit direkt voraus. Für die Solverauswahl müssen wir jedoch nur wissen, welcher Solver am schnellsten ist – nicht die genaue Laufzeit. Diese Arbeit konzentriert sich auf die Genauigkeit der Solverauswahl.

2.2.3 Training und Prediction Pipeline

Die gesamte Satzilla-Pipeline besteht aus:

$$
\begin{tikzcd}
\text{CNF File} \arrow[r] & \text{Feature Extractor} \arrow[r] & \text{Features} \arrow[d] \\
 & & \text{Trained Model} \arrow[d] \\
\text{Runtime Data (training)} \arrow[r] & \text{Training} \arrow[r] & \text{Predictions (which solver is fastest)}
\end{tikzcd}
$$

1. Training Phase: Gegeben sei eine Menge von Trainingsinstanzen mit bekannten Laufzeiten für jeden Solver:

- Features aus jeder CNF-Datei extrahieren
- Ermitteln Sie für jede Instanz, welcher Solver am schnellsten war
- Trainieren Sie einen Random-Forest-Klassifikator, um den schnellsten Solver vorherzusagen

2. Prediction Phase: Für eine neue CNF-Datei:

- Die gleichen Features extrahieren
- Wende das trainierte Modell an, um den schnellsten Solver vorherzusagen
- Führen Sie nur diesen Solver aus (das spart Zeit)

2.2.4 Warum Satzilla nach wie vor von Bedeutung ist

Obwohl Satzilla bereits 2008 eingeführt wurde, sind seine Grundsätze nach wie vor aktuell:

- Rechenintensivitätsarme Merkmale (Sekunden pro Instanz) im Vergleich zu rechenintensiven Solver-Läufen (Minuten bis Stunden)
- Einmalige Trainingskosten, die über viele Vorhersagen verteilt abgeschrieben werden
- Transparent und nachvollziehbar (die Merkmalsbedeutung zeigt, worauf es ankommt)
- Anpassbar an neue Solver-Sätze (wie in dieser Arbeit gezeigt)

3 Satzilla Outcome

3.1 Validierung mit Fake Solvern

Da ich in meinem ursprünglichen Bachelorprojekt Predictions in Satzilla hatte die nicht mit den tatsächlichen vorhersagen übereinstimmten (so wurde fälschlicherweiße ISAsat alle der beste Solver für viele Probleme empfohlen) riet mir mein Tutor Dr. Mathias Fleury zu testen ob Satzilla überhaupt richtig funktioniert und passende vorhersagen machen kann.

Bevor ich Satzilla auf die Laufzeitdaten realer Solver angewandt haben, habe ich zunächst die gesamte Pipeline anhand künstlich erzeugter Solver-Verhaltensmuster validiert. Dieser Schritt war entscheidend, um sicherzustellen, dass die Merkmalsextraktion, die Datenausrichtung und der Lernmechanismus korrekt funktionieren.

3.1.1 Design der fake Solver

Ich habe künstliche Solver erstellt, die explizite, bekannte Beziehungen zu den Eigenschaften der Instanzen aufweisen (z. B. zur Anzahl der Variablen). Auf diese Weise konnte ich eine Referenzgröße definieren: Für jede Instanz wussten ich genau, welcher Solver als der schnellste anzusehen war.

- **3-Solver-Test**: Auf kleine, mittlere und große Instanzen spezialisierte Solver, basierend auf Schwellenwerten für die Variablenanzahl (33. und 67. Perzentil).
- **4-Solver-Test**: Zwei Solver basieren auf der Variablenanzahl und zwei auf der Klauselanzahl, wobei eine klare Hierarchie besteht (Variablenspezialisten dominieren).

3.1.2 Resultate

Genauigkeit der Ergebnisse der Fake Solver:

Test	Solver Count	Accuracy
Sprungfunktion mit beliebigen Schwellenwerten (100k/200k)	3	65%
Lineare Funktion (laufzeit in relation zur Anzahl der Variablen)	3	99%
Sprungfunktion mit datenabhängigen Schwellenwerten	3	99.25%
Schrittweise Hierarchie (Variablen + Klauseln)	4	99.75%

Table 4: Testergebnisse für verschiedene Auswahlstrategien

Wichtigste Features der Fake Solver:

Rang	Feature	Relevanz
1	nvarsOrig	13.07%
2	VCG-CLAUSE-min	8.46%
3	nvars	7.79%

Table 5: Testergebnisse der wichtigsten Features

3.1.3 Interpretation

Die hohe Genauigkeit ($\geq 99\%$) in drei von vier Tests bestätigte, dass:

- Die Feature extraction erfasst die Eigenschaften der Instanzen korrekt. nvarsOrig führt die direkte Messung der Anzahl der ursprünglichen Variablen durch – das wichtigste Unterscheidungsmerkmal, nvars die Anzahl der Variablen nach dem Preprocesing und VCG-CLAUSE-min den minimaler Klauselgrad in einem Graphen mit variablen Klauseln (korreliert mit der Größe)
- Die Trainingspipeline (Random-Forest-Klassifikator, Kreuzvalidierung) funktioniert wie vorgesehen.
- Das Scheitern des ersten Tests (65 %) war auf schlecht gewählte Schwellenwerte (100k/200k) zurückzuführen, die zu einem unausgewogenen Datensatz führten – nicht auf einen Fehler in Satzilla selbst.

Fazit: Die Satzilla-Pipeline funktioniert einwandfrei und ist bereit für das Training mit echten Solver-Daten.

3.2 Trainieren auf echten Kissat Solvern

3.2.1 Dataset

3.2.2 Training Konfiguration

3.2.3 Testgenauigkeit

3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“

3.3.1 Resultate

3.3.2 Interpretation

3.3.3 Kumulative Genauigkeitskurve

3.4 Feature Importance Analysis

3.4.1 Wichtigste Features

3.4.2 Konklusion

3.5 Per-Solver-Leistung

3.5.1 Interpretation

3.6 Vergleich mit einer zufälligen Referenzlinie

3.7 Praktischer Nutzen für die SAT-Lösung

3.8 Auswirkungen der Hinzufügung von fehenden LP-Features

3.8.1 Interpretation

3.9 Zusammenfassung der wichtigsten Ergebnisse

3.10 Konklusion

4 Results

5 Discussion

6 Conclusion

ToDos

Select the correct language in the first rows of `main.tex`. The % makes a line a comment.

Generate a PDF (Visual Studio Code: click on the green arrow in the top right corner).

Check the PDF (Visual Studio Code: click on the two rectangles with the magnifying glass in the top right corner).

Talk about this template with your supervisor: Should you add a declaration for the use of AI? Currently no official AI-declaration is given. You can find one in the pull requests of the github repository.

With CTRL pressed you can click on the PDF and jump to the corresponding place in your `.tex` files.

Check the Errors. (Visual Studio Code: bottom left) There should be one warning because the first ToDo pushes down the second one: `LaTeX Marginpar on page 2 moved.`

In the `main.tex` you can press ALT + SHIFT and the ARROW UP and DOWN keys to create a multiline cursor, comment out all the sections you are currently not working on and remove the multiline cursor with ESC.

Keep the following examples and todos till the end and check them out sometimes.

Adjust your License Notes to reflect the used programs.

For Printing: Remove the `todonotes`-package in `setup.tex` to find all ToDos.

Table 6: A nice table with a nice caption.

Value & Unit	Value	Unit

For Printing: Choose the correct formating (search `print` in `setup.tex`).

You can remove the glossary and the symbolslist to make it easier, especially in a Bachelor's Thesis.

6.1 Tables and References

Table 6

This is a cited book [2].

Store and display plots and other graphs as `.pdf` files.

Include Algorithm 1 in the same way you include equations like Equation 1:

$$\begin{aligned} y &= 3x^2 + 5x^2 + 4x + 4x + 7 + 1 \\ &= 8x^2 + 8x + 8 \\ y &= 8 \cdot (x^2 + x + 1) \end{aligned} \tag{1}$$

6.2 Auto-complete in VSCode

There are also some auto-complete features: type @ + letter to get the letters greek version.

Type @v + letter for the alternative form (like φ for ϕ).

You can also use shortcuts for environments like

$$BAL \tag{2}$$

Type BAL and select the correct option from the autocomplete. Maybe you have to try various ones. The B is the base command and AL is the abbreviation for align.

If you want an environment without an index, you can use BS as base command.

Same for changes to your font, like bold or italic: FIT gives *text*

6.3 ", Quotations and Code

Use " or `file-names`, `file_names` and `code_snippets`.

Algorithm 1: The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in `setup.tex`.

```
from dataclasses import dataclass

# https://cd.uni-freiburg.de/farben/

@dataclass
class MainColors:
    blue = "#344A9A"
    black = "#000000"
    white = "#FFFFFF"

@dataclass
class BackgroundColors:
    blue_100_percent = "#344A9A"
    blue_80_percent = "#5D6BAD"
    blue_60_percent = "#868DC2"
    blue_80_percent = "#AFB1D8"
    blue_20_percent = "#D7D8EC"
    dark_blue = "#00004a"
    sand_100_percent = "#f6f1e3"
    sand_50_percent = "#faf8f1"

@dataclass
class AdditionalColors:
    green_100_percent = "#00a082"
    green_80_percent = "#27b29b"
    green_60_percent = "#7bc6b4"
```

```
green_80_percent = "#afdace"
green_20_percent = "#daede7"
brown_100_percent = "#8f6b30"
brown_80_percent = "#a58959"
brown_60_percent = "#bca683"
brown_80_percent = "#d2c4ac"
brown_20_percent = "#e9e1d6"
yellow_100_percent = "#ffe863"
yellow_80_percent = "#ffed82"
yellow_60_percent = "#fff1a1"
yellow_80_percent = "#fff6c1"
yellow_20_percent = "#fffae0"
pink_100_percent = "#f5c2ed"
pink_80_percent = "#f7cef1"
pink_60_percent = "#f9daf4"
pink_80_percent = "#fbe7f8"
pink_20_percent = "#fdf3fb"
```

6.4 Clipboard Manager, Git and more

Clipboard Manager help with copying multiple different things at once. On Windows you can press WINDOWS + V to activate the history and choose older things you copied.

Use Git (click here).

6.5 Glossary

Click them, the page numbers in the glossaries are also clickable.

name one

name one

name one

long form of name two (short2)

short2

short2

$a\ symbol$

$a\ symbol$

$a\ symbol$

Big Letter at start: Name one

plural of name one

name one

Big Letter at start: Short2

short2

short2s

Big Letter at start: $A\ symbol$

$a\ symbol$

Symbols

Notation	Description	Page List
$a\ symbol$	description symbol	18, 19

List of Figures

List of Tables

1 Kissat Optimierungen 4
2 Kissat-Konfigurationsvarianten 5
3 SAT Instance Features Categories 6
4 Testergebnisse für verschiedene Auswahlstrategien 9
5 Testergebnisse der wichtigsten Features 10
6 A nice table with a nice caption. 15

List of Algorithms

1 The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in `setup.tex`. 16

Glossary

Notation	Description	Page List
name one	description one	17, 19
short2	description two	17–19

References

[1] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. Cadical, kissat, paracooba, plingeling and treengeling entering the sat competition 2020. In *Proc. of SAT Competition 2020 - Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 50–53, Helsinki, 2020. University of Helsinki.

[2] Leander Bürkin. *LaTeX-Template*. Fachschaft TF Freiburg, 2025.

[3] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, New York, NY, USA, 1971. ACM.

[4] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla: Portfolio-based algorithm selection for sat. *Journal of Artificial Intelligence Research (JAIR)*, 32:565–606, 2008.

Appendix